

WISHMATE · TECHNICAL NOTE

React Router와 Next.js 16 App Router 비교

라우트 정의, 렌더링 방식, 데이터 흐름과 History API의 관계를
중심으로 정리한 기술 문서

대상 프로젝트 WishMate

기준 버전 Next.js 16.3.5

작성일 2026-09-18

한눈에 보는 핵심

둘 다 URL에 맞는 화면을 보여 주는 라우터이지만 역할 범위가 다르다.

- **React Router**: React 애플리케이션에 라우팅 기능을 추가하는 독립 도구다. 현재는 단순 라우터뿐 아니라 데이터 로딩과 SSR까지 제공하는 프레임워크 모드도 있다.
- **Next.js App Router**: Next.js 전체 프레임워크에 내장된 라우팅·서버 렌더링·데이터 처리 시스템이다.

1. React Router

React Router의 기본 역할은 다음과 같다.

URL과 React 컴포넌트를 연결하고, 새로고침 없이 화면을 전환한다.

전통적인 선언형 사용 방식은 다음과 같다.

JSX

```
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
} from "react-router";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/"> 홈 </Link>
        <Link to="/products"> 상품 </Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}
```

각 요소의 역할은 다음과 같다.

- **<BrowserRouter>**: 브라우저 History API와 React 애플리케이션을 연결한다.
- **<Routes>**: 현재 URL과 일치하는 라우트를 탐색한다.
- **<Route>**: URL과 컴포넌트의 대응 관계를 정의한다.
- **<Link>**: 새로고침 없는 페이지 이동을 제공한다.
- **useNavigate()**: 코드에서 명령형으로 이동한다.
- **useParams()**: `/products/:id`의 `id`처럼 동적 경로 값을 확인한다.
- **<Outlet>**: 중첩 라우트의 자식 화면을 표시한다.

React Router는 현재 세 가지 사용 모드를 제공한다.

1.1 Declarative Mode

가장 전통적인 사용 방식이다.

JSX

```
<BrowserRouter>
  <Routes>
    <Route path="/products" element={<Products />} />
  </Routes>
</BrowserRouter>
```

React Router는 주로 URL과 화면 연결을 담당하고, 데이터 요청은 [fetch](#), React Query, Zustand 등 별도 도구로 처리한다.

1.2 Data Mode

라우트에 데이터 로딩과 변경 로직을 연결할 수 있다.

JSX

```
const router = createBrowserRouter([
  {
    path: "/products/:id",
    Component: ProductDetail,
    loader: loadProduct,
    action: updateProduct,
  },
]);
```

- [loader](#): 화면에 필요한 데이터를 조회한다.
- [action](#): 등록·수정·삭제 같은 데이터 변경을 처리한다.
- [useFetcher](#): 페이지 이동 없이 데이터를 조회하거나 변경한다.
- pending/error 상태를 라우팅 흐름에 맞춰 관리할 수 있다.

1.3 Framework Mode

React Router를 단순 라이브러리가 아니라 전체 웹 프레임워크처럼 사용한다.

- Vite 플러그인
- Route Module
- 코드 분할
- SPA 렌더링
- SSR
- 정적 사전 렌더링
- 서버 [loader](#)와 [action](#)

라우트는 기본적으로 설정 파일에서 정의한다. 별도 `@react-router/fs-routes` 패키지를 사용하면 파일 시스템 기반 라우팅도 사용할 수 있다.

따라서 “React Router는 무조건 브라우저에서만 동작하는 SPA 라우터”라는 설명은 현재 기준으로는 정확하지 않다. Framework Mode에서는 SSR과 정적 렌더링도 가능하다.

다만 React Server Components 지원은 현재 실험적 기능이며 기본 아키텍처는 아니다.

2. Next.js 16 App Router

Next.js App Router는 `app` 디렉터리의 폴더와 파일 구조로 URL을 정의한다.

TEXT

```
app/
├── layout.js
├── page.js
├── products/
│   ├── page.js
│   └── [id]/
│       └── page.js
└── seller/
    └── products/
        └── page.js
```

이 구조는 다음 URL이 된다.

TEXT

```
app/page.js           → /
app/products/page.js  → /products
app/products/[id]/page.js → /products/123
app/seller/products/page.js → /seller/products
```

폴더가 URL 세그먼트가 되고, `page.js`가 해당 URL에서 표시할 화면이 된다.

2.1 특수 파일

App Router는 파일 이름 자체에 기능이 부여된다.

파일	역할
<code>page.js</code>	해당 경로의 페이지
<code>layout.js</code>	하위 경로가 공유하는 레이아웃
<code>loading.js</code>	페이지 이동·조회 중 로딩 UI
<code>error.js</code>	라우트 단위 오류 처리
<code>not-found.js</code>	404 UI
<code>route.js</code>	HTTP API 엔드포인트
<code>[id]</code> 폴더	동적 URL 세그먼트

파일	역할
(auth) 폴더	URL에 포함되지 않는 Route Group

`layout.js`는 페이지가 바뀌어도 유지될 수 있기 때문에 헤더, 메뉴, 사이드바 같은 공통 UI에 적합하다.

2.2 Server Component가 기본

App Router의 중요한 특징은 `page.js`와 `layout.js`가 기본적으로 **Server Component**라는 점이다.

JSX

```
export default async function ProductPage({ params }) {
  const { id } = await params;
  const product = await getProductById(id);

  return <h1>{product.name}</h1>;
}
```

Server Component에서는 다음 작업을 직접 할 수 있다.

- 데이터베이스 조회
- 서버 전용 라이브러리 사용
- 인증 정보 확인
- 비밀 키 사용
- `async/await` 데이터 로딩

이 코드는 브라우저 JavaScript 번들에 그대로 포함되지 않는다.

클릭 이벤트, 상태, 브라우저 API 등이 필요할 때만 `"use client"`를 사용한다.

JSX

```
"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount((value) => value + 1)}>
      {count}
    </button>
  );
}
```

Next.js는 Server Component를 기본으로 하고 필요한 영역만 Client Component로 만드는 서버 우선 구조다.

2.3 페이지 이동

기본 이동은 `next/link`의 `<Link>`를 사용한다.

JSX

```
import Link from "next/link";

<Link href="/products/123">상품 보기</Link>
```

명령형 이동이 필요하다면 Client Component에서 `useRouter()`를 사용할 수 있다.

JSX

```
"use client";

import { useRouter } from "next/navigation";

function Button() {
  const router = useRouter();

  return (
    <button onClick={() => router.push("/products")}>
      상품으로 이동
    </button>
  );
}
```

서버에서는 `redirect()`를 사용할 수 있다.

JSX

```
import { redirect } from "next/navigation";

export async function createProductAction(formData) {
  const product = await createProduct(formData);
  redirect(`/products/${product.id}`);
}
```

`<Link>` 이동 시 Next.js는 공유 레이아웃을 유지하면서 필요한 새 라우트 데이터만 받아 화면을 교체한다. 또한 링크 사전 로딩과 클라이언트 전환을 지원한다.

Next.js 16에서는 내부 라우팅과 내비게이션 시스템이 다시 설계되어, 이미 캐시에 있는 부분을 제외한 필요한 부분만 점진적으로 prefetch하는 방식이 도입되었다. 애플리케이션에서 사용하는 `Link`, `useRouter`, `redirect`라는 기본 모델은 유지된다.

3. React Router와 Next.js App Router 비교

비교 항목	React Router	Next.js 16 App Router
정체	독립적인 React 라우터, 선택적으로 프레임워크	Next.js에 내장된 전체 라우팅 시스템
설치	별도 패키지 설치	Next.js에 기본 포함
라우트 정의	JSX 또는 설정 객체, Framework Mode에서는 route config	app 폴더와 page.js 파일
동적 경로	/products/:id	app/products/[id]/page.js
중첩 화면	<Outlet>	중첩 layout.js
링크	<Link to="/products">	<Link href="/products">
명령형 이동	useNavigate()	useRouter().push()
서버 이동	redirect()	redirect()
기본 렌더링 성격	모드와 설정에 따라 CSR-SSR-정적 렌더링	Server Component 중심의 서버 우선 렌더링
데이터 조회	외부 라이브러리 또는 loader	async Server Component에서 직접 조회
데이터 변경	action, <Form>, useFetcher	Server Action, <form action>
API 구현	별도 서버 또는 Framework Mode 서버	route.js Route Handler
RSC	실험적 지원	핵심 기본 구조
클라이언트 JS	일반적으로 라우트 컴포넌트를 hydrate	Server Component는 클라이언트 JS에 포함되지 않음
구성 자유도	높음	Next.js 규칙을 따라야 함
적합한 경우	기존 React SPA, 자체 서버-번들러 구성	서버 렌더링 중심의 풀스택 React 프로젝트

4. 가장 큰 개념적 차이

React Router의 기본 사고방식은 다음에 가깝다.

TEXT

URL 변경
→ 일치하는 React 컴포넌트 선택
→ 브라우저에서 해당 컴포넌트 렌더링

Next.js App Router는 다음에 가깝다.

TEXT

URL 변경

- Next.js 서버에서 해당 라우트와 데이터 처리
- Server Component 결과 생성
- 필요한 결과를 브라우저로 전달
- 기존 레이아웃을 유지하며 화면 갱신

React Router에서 "라우팅"은 비교적 독립적인 한 기능이다. Next.js App Router에서 "라우팅"은 다음을 함께 결정한다.

- 어떤 Server Component를 실행할지
- 어떤 레이아웃을 유지할지
- 어디서 데이터를 조회할지
- 어떤 로딩·오류 화면을 사용할지
- 어떤 코드를 브라우저에 전달할지
- 어떤 데이터를 사전 로딩할지

5. History API와의 관계

두 라우터 모두 브라우저의 History API를 내부적으로 활용하지만, 직접 `pushState()`만 호출하는 것과는 차이가 있다.

JS

```
window.history.pushState(null, "", "/products");
```

위 코드는 URL만 변경한다. React Router나 Next.js에는 "새 라우트를 렌더링하라"는 정보를 직접 전달하지 않는다.

반면 다음 API들은 URL뿐 아니라 라우터 상태와 화면을 함께 갱신한다.

JSX

```
// React Router
<Link to="/products" />
navigate("/products");

// Next.js
<Link href="/products" />
router.push("/products");
```

따라서 실제 페이지 이동에는 각 라우터의 API를 사용하고, History API 직접 호출은 현재 페이지의 검색 조건처럼 제한적인 URL 동기화에 사용하는 것이 일반적이다.

6. 현재 Wishlist 프로젝트에 적용하면

현재 프로젝트는 `package.json`에 Next.js 16.3.5가 설치되어 있고 React Router는 설치되어 있지 않다.

또한 다음 App Router 구조를 사용한다.

- 루트 레이아웃: [app/layout.js](#)
- 페이지 이동용 [next/link](#): [components/site-header.js](#)
- 서버 처리 후 [redirect\(\)](#): [app/group-gifts/actions.js](#)
- 필터 URL만 변경하는 History API: [app/home-product-board.js](#)

따라서 이 프로젝트에서는 **React Router**를 추가할 이유가 없으며 **Next.js App Router**가 주 라우터다. React Router를 함께 도입하면 URL 처리 체계가 두 개가 되어 구조와 상태 동기화만 복잡해질 가능성이 크다.

현재 [history.pushState\(\)](#) 사용은 별도 라우터가 아니라, 같은 페이지 안에서 필터 조건을 URL에 기록하기 위한 예외적인 보조 방식이다.

참고 자료

- [React Router: Picking a Mode](#)
- [React Router: Framework Routing](#)
- [React Router: Rendering Strategies](#)
- [React Router: React Server Components](#)
- [Next.js: Layouts and Pages](#)
- [Next.js: Server and Client Components](#)
- [Next.js: Linking and Navigating](#)
- [Next.js: Version 16](#)